Լաբորատոր աշխատանք 9

Դասի դինամիկ անդամներով խնդիրներ՝ մաթեմատիկական անալիզի թեմաներով

Աշխատանքի նպատակ. Սովորել C++ ծրագրավորման լեզվում ստեղծել դասեր, որոնք օգտագործում են դինամիկ զանգվածներ՝ տվյալների պահպանման և մշակման համար: Ծանոթանալ պարամետրերով կոնստրուկտորի, պատճենող կոնստրուկտորի, վերագրման օպերատորի և դեստրուկտորի կիրառմանը՝ պահպանելով Rule of Three սկզբունքը: Զարգացնել խորքային պատճենման, դինամիկ հիշողության կառավարման, Էյլերի մեթոդով դիֆերենցիալ հավասարումների լուծման և հաշվարկված արժեքների վերլուծության հմտությունները:

Խնդրի պահանջ. Ստեղծել HomogeneousEquation դաս՝ համասեռ դիֆերենցիալ հավասարման լուծման համար: Դասի private բաժնում սահմանել երեք double* դինամիկ զանգված՝ x, v, y արժեքները պահպանելու համար, ինչպես նաև քայլերի քանակը՝ n, և քայլի երկարությունը՝ h: Կոնստրուկտորներն ու մեթոդները սահմանել public բաժնում: Տվյալների անունները գրել անգլերեն:

Տրված է հետևյալ համասեռ դիֆերենցիալ հավասարումը՝ $y' = 1 + y/x$, $y(1) = 2$:

Հավասարումը լուծելու համար անհրաժեշտ է կատարել $y = vx$ փոխարինումը, որի արդյունքում ստացվում է $v' = 1/x$ հավասարումը:

Հաշվարկներն իրականացնել Էյլերի մեթոդով՝ օգտագործելով h = 1 քայլը: Սկզբնական արժեքները սահմանել x = 1, y = 2, իսկ v = y/x:

Ծրագիրը պետք է օգտատիրոջից ստանա x-ի վերջնական արժեքը՝ 1-ից 50 միջակայքում: Մուտքագրված արժեքը պետք է լինի ամբողջ թիվ: Սխալ մուտքագրման դեպքում արտածել հասկանալի հաղորդագրություն և կրկին պահանջել արժեք:

Դասում իրականացնել պարամետրերով կոնստրուկտոր, պատճենող կոնստրուկտոր, վերագրման օպերատոր և դեստրուկտոր: Պատճենումը պետք է լինի խորքային, որպեսզի յուրաքանչյուր օբյեկտ ունենա սեփական դինամիկ զանգվածները:

Հաշվարկված արժեքները պահպանել դինամիկ զանգվածներում, արտածել աղյուսակի տեսքով և որոշել y-ի առավելագույն արժեքը:

Տեսական մաս.

- Սահմանում ենք HomogeneousEquation դասը՝ երեք դինամիկ զանգվածներով (x, v, y), քայլերի քանակով (n) և քայլի երկարությամբ (h):
- Ստեղծում ենք պարամետրերով կոնստրուկտոր, որը ստանում է x-ի վերջնական արժեքը, որոշում քայլերի քանակը և դինամիկ հիշողություն հատկացնում զանգվածներին:
- Սահմանում ենք սկզբնական արժեքները՝ x[0] = 1, y[0] = 2, v[0] = 2:
- Իրականացնում ենք պատճենող կոնստրուկտոր՝ ապահովելով զանգվածների խորքային պատճենումը:

- Իրականացնում ենք վերագրման օպերատոր, որը պատճենում է մեկ օբյեկտի տվյալները մյուսի մեջ՝ ազատելով նախկինում հատկացված հիշողությունը:
- Ստեղծում ենք դեստրուկտոր՝ դինամիկ հիշողությունն ազատելու համար:
- Կատարում ենք y = vx փոխարինումը: Քանի որ y′ = v + xv′, ապա սկզբնական y′ = 1 + y/x հավասարումից ստանում ենք v′ = 1/x:
- Ստեղծում ենք calculate() մեթոդը, որը Էյլերի մեթոդով հաշվում է հաջորդական արժեքները՝ x[i+1] = x[i] + h, v[i+1] = v[i] + h/x[i], y[i+1] = v[i+1] * x[i+1]:
- Ստեղծում ենք findMax() մեթոդը՝ y զանգվածի առավելագույն արժեքը գտնելու համար:
- Ստեղծում ենք show() մեթոդը՝ հաշվարկված արժեքներն աղյուսակով արտածելու համար:
- main() ֆունկցիայում ստուգում ենք օգտատիրոջ մուտքագրած արժեքը՝ թույլատրելով միայն 1-ից 50 ամբողջ թվերը:
- Ստեղծում ենք առաջին օբյեկտը և կատարում հաշվարկները:
- Ստեղծում ենք երկրորդ օբյեկտը պատճենող կոնստրուկտորով և ստուգում խորքային պատճենումը:
- Ստեղծում ենք երրորդ անկախ օբյեկտը և ստուգում վերագրման օպերատորի աշխատանքը:
- Ծրագիրը պետք է էկրանին արտածի միայն հաշվարկված արժեքների աղյուսակը և y-ի առավելագույն արժեքը:
- Կոդում ավելացնում ենք հայերեն մեկնաբանություններ և կիրառում հասկանալի հաղորդագրություններ:

Առաջադրանք. Կազմել C++ ծրագիր, որը կկիրառի HomogeneousEquation դասը՝ y′ = 1 + y/x, y(1) = 2 համասեռ դիֆերենցիալ հավասարումը Էյլերի մեթոդով լուծելու համար: Ծրագիրը պետք է օգտագործի դինամիկ զանգվածներ՝ x, v և y արժեքները պահպանելու նպատակով: Անհրաժեշտ է իրականացնել պարամետրերով կոնստրուկտոր, պատճենող կոնստրուկտոր, վերագրման օպերատոր և դեստրուկտոր՝ պահպանելով Rule of Three սկզբունքը:

Ծրագիրը պետք է օգտատիրոջից ստանա x-ի վերջնական արժեքը՝ 1-ից 50 միջակայքում, կատարի հաշվարկները x = 1 սկզբնական արժեքից մինչև մուտքագրված արժեքը՝ h = 1 քայլով, և արդյունքները ցուցադրի աղյուսակով: Վերջում անհրաժեշտ է որոշել և արտածել y-ի առավելագույն արժեքը: Պետք է նաև ապահովել մուտքային տվյալների ստուգումը և օբյեկտների խորքային պատճենումը

```
#include <iostream>
using namespace std;

class HomogeneousEquation {
private:
   double* x;
   double* v;
   double* y;
   int n;
   double h;
public:
   // Պարամետրերով կոնստրուկտոր
```

```
HomogeneousEquation(int endX) {
    h = 1;
    n = endX - 1;
    x = new double[n + 1];
    v = new double[n + 1];
    y = new double[n + 1];
    x[0] = 1;
    y[0] = 2;
    v[0] = y[0] / x[0];
    for (int i = 1; i <= n; i++) {
        x[i] = 0;
        v[i] = 0;
        y[i] = 0;
    }
}
// Պատճենող կոնստրուկտոր
HomogeneousEquation(const HomogeneousEquation& other) {
    n = other.n;
    h = other.h;
    x = new double[n + 1];
    v = new double[n + 1];
    y = new double[n + 1];
    // Խորքային պատճենում
    for (int i = 0; i <= n; i++) {
        x[i] = other.x[i];
        v[i] = other.v[i];
        y[i] = other.y[i];
    }
}
// Վերագրման օպերատոր
HomogeneousEquation& operator=(const HomogeneousEquation& other) {
    if (this != &other) {
        double* newX = new double[other.n + 1];
        double* newV = new double[other.n + 1];
        double* newY = new double[other.n + 1];
        for (int i = 0; i <= other.n; i++) {
            newX[i] = other.x[i];
            newV[i] = other.v[i];
            newY[i] = other.y[i];
        }
        delete[] x;
        delete[] v;
        delete[] y;
        x = newX;
        v = newV;
        y = newY;
```

```
        n = other.n;
        h = other.h;
    }
    return *this;
}
// Դեստրուկտոր
~HomogeneousEquation() {
    delete[] x;
    delete[] v;
    delete[] y;
}
// Էյլերի մեթոդով հաշվարկ
void calculate() {
    for (int i = 0; i < n; i++) {
        x[i + 1] = x[i] + h;
        v[i + 1] = v[i] + h / x[i];
        y[i + 1] = v[i + 1] * x[i + 1];
    }
}
// Y-ի առավելագույն արժեքի որոնում
double findMax() const {
    double maxY = y[0];
    for (int i = 1; i <= n; i++) {
        if (y[i] > maxY) {
            maxY = y[i];
        }
    }
    return maxY;
}
// Աղյուսակի արտածում
void show() const {
    cout << "\nՔայլ\tX\tV\tY\n";
    cout << "----------------------------------------\n";
    for (int i = 0; i <= n; i++) {
        cout << i << "\t"
             << x[i] << "\t"
             << v[i] << "\t"
             << y[i] << endl;
    }
    cout << "\nY-ի առավելագույն արժեքը = "
         << findMax() << endl;
}
// Արժեքի փոփոխում՝ խորքային պատճենման ստուգման համար
void changeFirstValue(double value) {
    y[0] = value;
}
```

```
    // Առաջին արժեքի վերադարձ
    double getFirstValue() const {
        return y[0];
    }
};
int main() {
    int endX;
    cout << "Համասեռ դիֆերենցիալ հավասարում\n";
    cout << "y' = 1 + y/x, y(1) = 2\n";
    cout << "Քայլը՝ h = 1\n";
    // Մուտքագրված արժեքի ստուգում
    while (true) {
        cout << "\nՄուտքագրեք X-ի վերջնական արժեքը (1-50): ";
        string input;
        cin >> input;
        // Ստուգում ենք, որ մուտքագրվածը ամբողջ թիվ լինի
        bool valid = !input.empty();
        int number = 0;
        for (int i = 0; i < static_cast<int>(input.length()); i++) {
            if (input[i] < '0' || input[i] > '9') {
                valid = false;
                break;
            }
            // Թիվը հավաքում ենք միայն թույլատրելի միջակայքում
            number = number * 10 + (input[i] - '0');
            if (number > 50) {
                valid = false;
                break;
            }
        }
        if (!valid || number < 1) {
            cout << "Սխալ: Մուտքագրեք ամբողջ թիվ 1-ից 50:\n";
            continue;
        }
        endX = number;
        break;
    }
    // Առաջին օբյեկտի ստեղծում և հաշվարկ
    HomogeneousEquation obj1(endX);
    obj1.calculate();
    // Արդյունքների արտածում
    obj1.show();
    // Խորքային պատճենման ստուգում՝ առանց արտածման
    HomogeneousEquation obj2 = obj1;
    obj2.changeFirstValue(100);
```

```
    // Երկրորդ անկախ օբյեկտ և վերագրման ստուգում
    HomogeneousEquation obj3(endX);
    obj3.calculate();
    obj3 = obj1;
    obj3.changeFirstValue(200);

    return 0;
}
```